Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche et analyse de complexité

Synthèse exécutive : Ce cours aborde les méthodes fondamentales pour retrouver une valeur au sein d'une structure de données linéaire (tableau). L'enjeu central repose sur la relation directe entre l'organisation des données (tableau trié ou non trié) et la performance temporelle de l'algorithme choisi. La recherche séquentielle parcourt le tableau de manière linéaire en O(n) sans aucune précondition sur les données. À l'inverse, la recherche dichotomique s'appuie sur le paradigme 'diviser pour régner' pour réduire l'espace de recherche de moitié à chaque étape, garantissant une complexité logarithmique en O(log n). L'étudiant apprend à maîtriser les deux algorithmes, l'analyse de leur complexité au pire des cas, la manipulation exacte des pointeurs d'indices (début, fin, milieu) et les critères d'arbitrage selon le contexte d'utilisation.

La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est la stratégie de recherche la plus intuitive et universelle. Elle consiste à passer en revue chaque élément du tableau successivement jusqu'à trouver la cible ou atteindre la fin du tableau. Si le tableau contient n éléments, le pire des cas se produit lorsque la valeur cherchée se trouve à la toute dernière position ou n'existe pas dans le tableau : l'algorithme effectue alors exactement n comparaisons. Sa complexité est donc linéaire O(n). Son principal avantage est son universalité : elle s'applique à n'importe quelle structure séquentielle sans nécessiter un tri préalable coûteux.

- Parcours exhaustif du tableau indice par indice, du début (0) jusqu'à la fin (n-1)
- Absence totale de précondition : fonctionne sur des tableaux triés ou non triés
- Complexité temporelle linéaire O(n) au pire des cas

Cas concret enseignant : *Le professeur utilise l'analogie de la recherche d'une clé tombée dans un tas de feuilles mortes. N'ayant aucun ordre ni rangement possible dans un tas de feuilles, il n'existe aucune astuce : la seule solution consiste à soulever chaque feuille une par une jusqu'à trouver la clé.*

La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique exploite l'ordre des éléments dans un tableau trié pour accélérer la recherche. À chaque étape, l'algorithme calcule l'indice du milieu : milieu = (debut + fin) // 2, puis compare la valeur T[milieu] à la cible recherchée. Si T[milieu] == cible, l'élément est trouvé. Si la cible est strictement inférieure à T[milieu], l'élément ne peut se trouver que dans la première moitié : le pointeur 'fin' devient 'milieu - 1'. Si elle est supérieure, on déplace 'debut' à 'milieu + 1'. En éliminant la moitié du tableau à chaque itération, l'espace de recherche décroît de façon exponentielle, permettant de trouver un élément parmi un million en environ 20 comparaisons seulement.

- Précondition absolue et stricte : Le tableau doit obligatoirement être trié au préalable
- Paradigme algorithmique 'Diviser pour Régner' réduisant l'espace de recherche de 50 % à chaque étape
- Gestion dynamique de 3 pointeurs d'indices : debut, fin et milieu = (debut + fin) // 2

Cas concret enseignant : *Le professeur illustre cela avec le jeu du 'Devine un nombre entre 1 et 100'. Proposer 1, 2, 3... est une approche séquentielle inefficace. Proposer 50 permet d'éliminer d'un coup 50 nombres si la réponse est 'c'est plus grand'. On trouve la solution en 7 étapes maximum (log2(100) ≈ 6,64).*

Arbitrage Algorithmique et Analyse de Performance

En informatique, le choix d'un algorithme dépend du contexte d'utilisation. Bien que la recherche dichotomique surpasse largement la recherche séquentielle, son coût caché réside dans la précondition de tri. Si un tableau n'est

pas trié, effectuer un tri rapide ou par fusion exige une complexité de O(n log n). Dès lors, pour effectuer une unique recherche ponctuelle sur un tableau non trié, il est plus efficient d'exécuter une recherche séquentielle en O(n). En revanche, dans le cas d'une base de données ou d'une structure statique interrogée de façon répétée (des milliers de recherches), procéder à un tri initial unique en O(n log n) devient un investissement extrêmement rentable, car chaque recherche ultérieure ne coûtera que O(log n).

- Coût du tri préalable : Trier un tableau non ordonné nécessite au mieux O(n log n)
- Recherche unique : La recherche séquentielle O(n) est plus rapide que de trier puis chercher
- Recherches multiples : Le coût du tri initial O(n log n) est largement amorti si le tableau subit de très nombreuses requêtes O(log n)

Cas concret enseignant : *Analogie du dictionnaire imprimé : La mise en ordre alphabétique des mots exige un travail colossal lors de l'impression initiale (le tri), mais cet effort unique permet ensuite à des millions de lecteurs de trouver un mot en quelques secondes durant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique impose une précondition absolue : le tableau doit être trié. Si le tableau n'est pas trié, le coût de son tri préalable en O(n log n) annule le gain de performance si l'on ne fait qu'une seule recherche.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière : milieu = (debut + fin) // 2. Le résultat est automatiquement arrondi à l'entier inférieur, ce qui fonctionne parfaitement pour séparer le sous-tableau en deux parties.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] composé de n = 10 éléments (indices 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur cible 42.

Démarche de résolution :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : - Calcul du milieu : milieu = (0 + 9) // 2 = 4. - Valeur examinée : T[4] = 19. - Comparaison : Cible 42 > 19. La valeur recherchée est strictement supérieure, elle se trouve donc dans la sous-partie droite. - Mise à jour du pointeur debut : debut = milieu + 1 = 5. Étape 2 : - Nouvelle zone de recherche : debut = 5, fin = 9. - Calcul du milieu : milieu = (5 + 9) // 2 = 7. - Valeur examinée : T[7] = 42. - Comparaison : T[7] == 42. La cible est identifiée ! - Succès de la recherche.

Résultat : La valeur 42 a été localisée à l'indice 7 du tableau en seulement 2 comparaisons grâce à la dichotomie, alors qu'une recherche séquentielle aurait nécessité 8 comparaisons (de T[0]=3 jusqu'à T[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative et indispensable pour pouvoir exécuter une recherche dichotomique sur un tableau ?

R : Le tableau doit être préalablement trié.

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle effectuée sur un tableau de n éléments ?

R : O(n)

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

R : 10 comparaisons au maximum (car log2(1024) = 10).